Learning objectives:

Review of linked lists

Introduction to queue.h

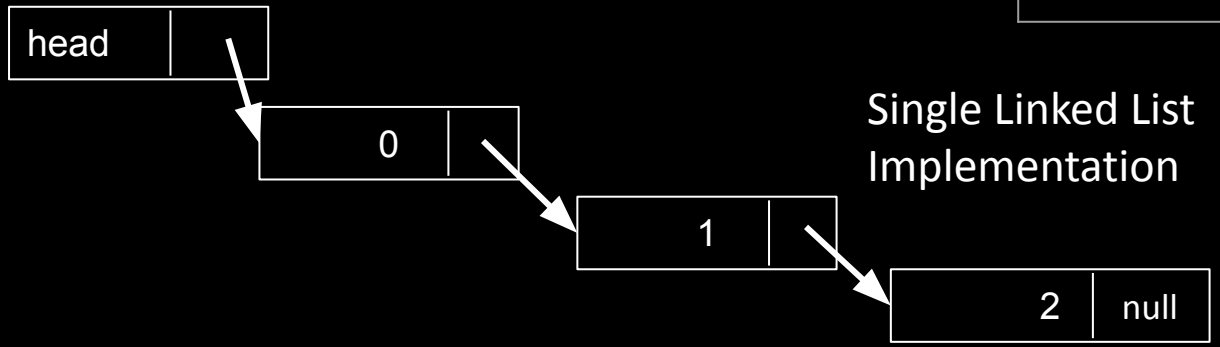
Threading suggestions for Assignment 6

Linked Lists

- Arrangement of values where each value is paired with a pointer to the next (and possibly previous value)
- Similar to an array but no requirement for sequential storage locations.

Array Implementation



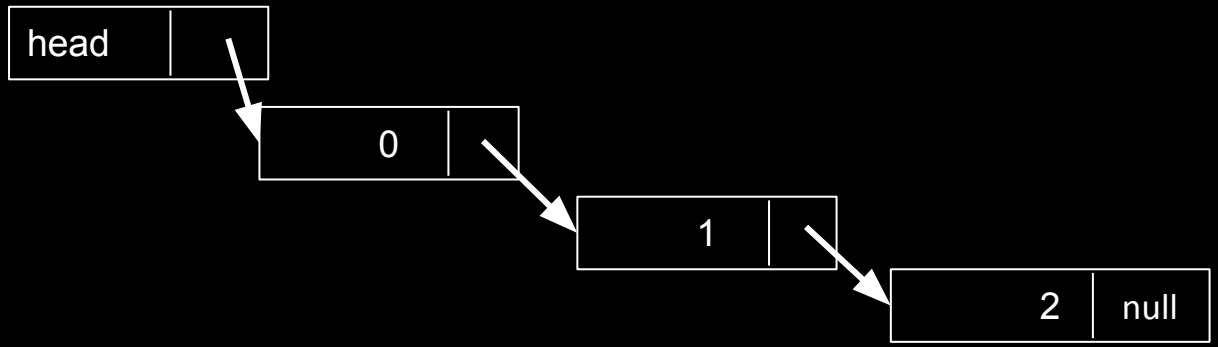


Single Linked List Implementation

http://cslibrary.stanford.edu/103/LinkedListBasics.pdf

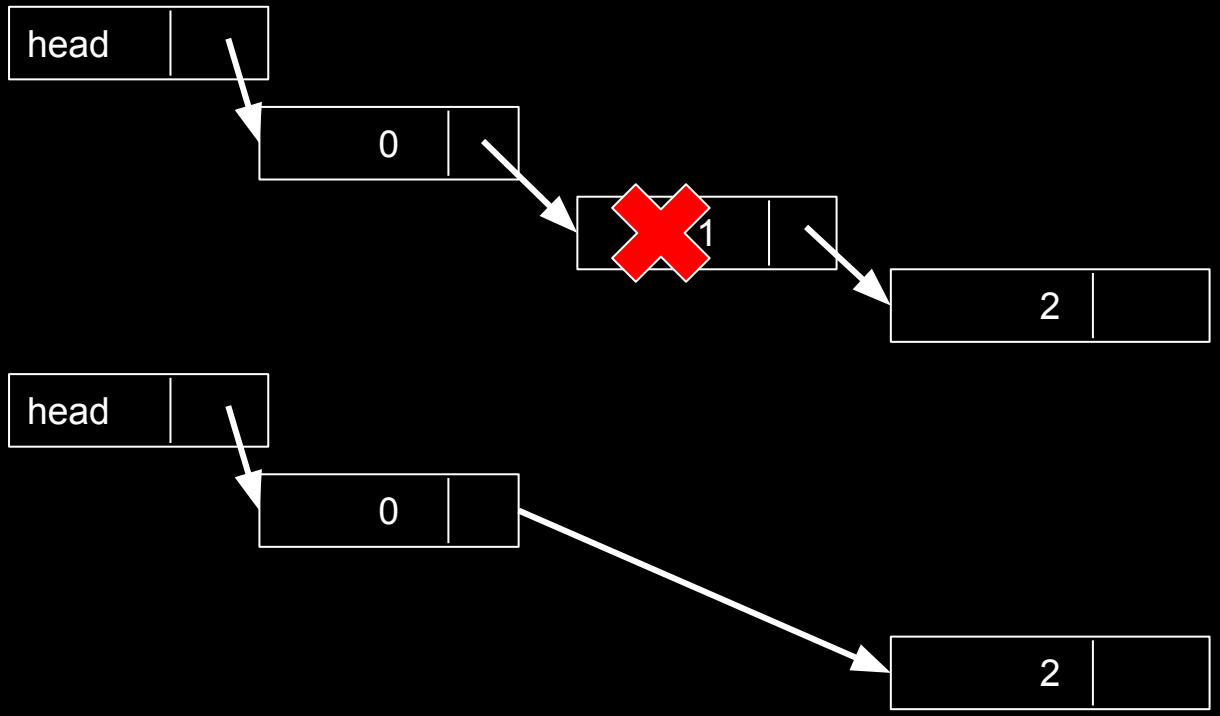
Linked Lists

- What are some benefits of Linked Lists over arrays?
  - Don’t need to know size in advance
  - Size of list is only limited by size of memory
  - Can add or remove nodes to sequence without moving memory



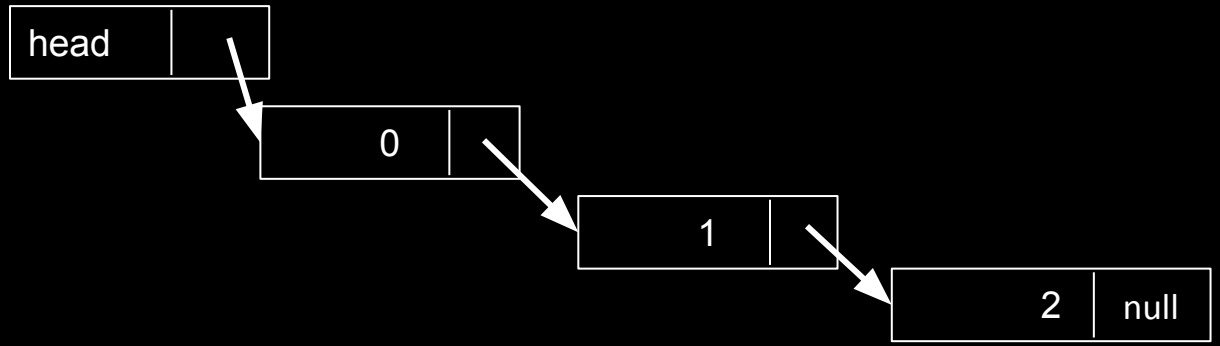
http://cslibrary.stanford.edu/103/LinkedListBasics.pdf

Linked List Removal



Linked List Removal

- What are some drawbacks of linked lists compared to arrays?
  - Extra memory usage for pointer to next element
  - Indexing (equivalent of value[offset]) is O(n)



Single Linked List Implementation

Implementing a Linked List

- Option 1: Implement yourself
  - Implementation steps which require special consideration:
    - removing the first element (head)
    - removing elements while iterating

Implementing a Linked List

- Option 2: Use an existing implementation
  - Benefit: Probably more thoroughly tested than your implementation would be
  - Should save time
  - Get familiar with using different APIs

queue.h

- C implementation of linked list can be found in queue.h
- SLIST = Singly Linked List
- SLIST_HEAD - declares your head element (first item in list)
- SLIST_INIT - initializes your head element
- SLIST_INSERT_HEAD - inserts an element at the head

https://raw.githubusercontent.com/freebsd/freebsd/stable/10/sys/sys/queue.h

https://github.com/stockrt/queue.h/blob/master/sample.c

queue.h

- SLIST_remove - REMOVE AN ELEMENT
- SLIST_FOREACH - iterate over the list
- SLIST_FOREACH_SAFE - iterate over the list, while allowing removal
  - Requires freebsd version of queue.h linked below

https://raw.githubusercontent.com/freebsd/freebsd/stable/10/sys/sys/queue.h

https://github.com/stockrt/queue.h/blob/master/sample.c

sample.c and queue.h

```c
struct slist_data_s {
    int value;
    SLIST_ENTRY(slist_data_s) entries;
};
```

```c
SLIST_HEAD(slisthead, slist_data_s) head;
```

```c
slist_data_t *datap=NULL;
```

```c
// Read1.
SLIST_FOREACH(datap, &head, entries) {
    printf("Read1: %d\n", datap->value);
}
printf("\n");
```

```c
#define SLIST_ENTRY(type)                                       \
struct {                                                        \
        struct type *sle_next;  /* next element */              \
}
```

```c
#define SLIST_HEAD(name, type)                                  \
struct name {                                                   \
        struct type *slh_first; /* first element */             \
}
```

```c
#define SLIST_FIRST(head)       ((head)->slh_first)
```

```c
#define SLIST_FOREACH(var, head, field)                         \
        for ((var) = SLIST_FIRST((head));                       \
            (var);                                              \
            (var) = SLIST_NEXT((var), field))
```

```c
#define SLIST_NEXT(elm, field)  ((elm)->field.sle_next)
```

https://github.com/stockrt/queue.h/blob/master/sample.c

https://raw.githubusercontent.com/freebsd/freebsd/stable/10/sys/sys/queue.h

Linked List/Threading in Assignment 6

- Assignment 6 adds threads to your socket server implementation
- Each socket connection should start a new thread after accept() to handle data transfers to/from
- How many threads should you support?
  - As many as you can with the memory you have.
  - Good application for a linked list!

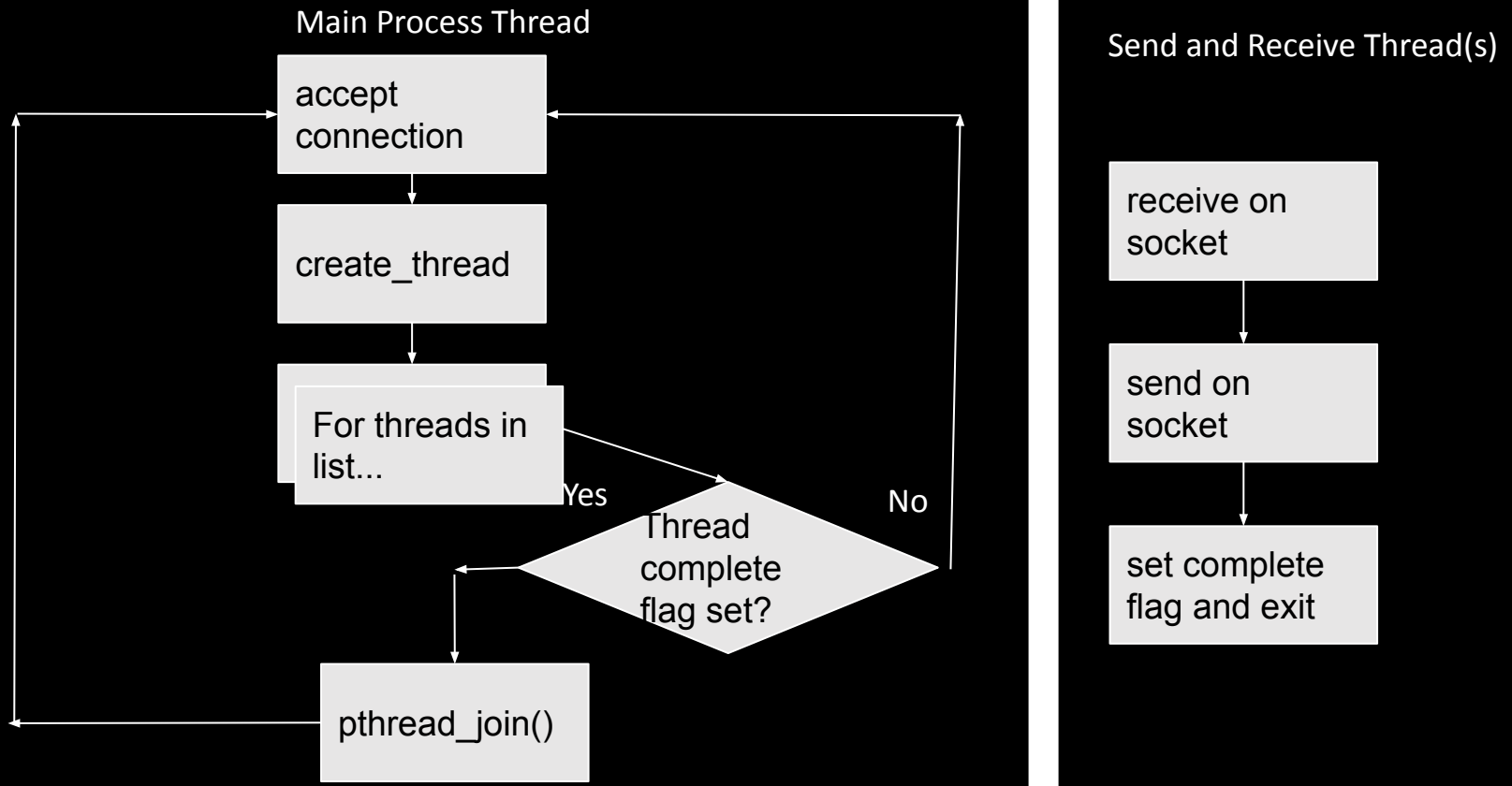
Linked List/Threading in Assignment 6

- You will need to track the thread IDs in your main process to:
- Check for completion (include a variable set by the thread)
  - You can do this each time you start a new thread in your main process
  - pthread_join() the thread ID to avoid memory leaks.

https://github.com/stockrt/queue.h/blob/master/sample.c

https://raw.githubusercontent.com/freebsd/freebsd/stable/10/sys/sys/queue.h

Threading Overview



Threading Pitfalls to Avoid

- Deallocate memory in only one place
  - Usually best to do this from the main process thread
- Keep in mind linked list implementation is not thread safe
  - Best to only access this from the main process thread

Threading Pitfalls to Avoid

- Don’t forget to pthread_join()
  - You will see an error from valgrind.
- Why can’t I just use detached threads?
  - Only safe to do this if you have some other synchronization method to know the thread has terminated.
  - Can often introduce race conditions on shutdown

https://stackoverflow.com/questions/22803600/when-should-i-use-stdthreaddetach